Residencial Golden Java - Sistema de Gestão Condominial

Trabalho N1 - Programação Orientada a Objetos

Grupo 02: Kaio Veras e Emanuel Veloso

1. Business Model Canvas



[Acessar documento original]

2. Telas do Sistema

Para representar as principais funcionalidades do sistema Residencial Golden Java, foram elaborados wireframes de baixa fidelidade. As telas apresentam a organização inicial da interface, os perfis de usuário e os fluxos essenciais do sistema, sem priorizar detalhes visuais ou elementos gráficos definitivos.

2.1 Perfis de Usuário

O sistema foi planejado considerando diferentes usuários, cada um com responsabilidades e permissões específicas.

Síndico

Responsável pela administração geral do condomínio. Pode consultar unidades e moradores, controlar pagamentos e despesas, acompanhar o saldo do condomínio, gerenciar funcionários e visualizar solicitações de manutenção.

Porteiro

Responsável pelo controle de acesso. Pode registrar a entrada e a saída de visitantes e prestadores, consultar autorizações e identificar a unidade de destino.

Chefe de manutenção

Responsável pelo acompanhamento dos serviços de manutenção. Pode consultar chamados, definir prioridades, atribuir responsáveis e atualizar o andamento ou a conclusão de cada serviço.

Morador

Pode consultar informações relacionadas à sua unidade, verificar taxas e pagamentos, autorizar visitantes e registrar solicitações de manutenção.

Funcionário

Representa os colaboradores do condomínio, como zeladores, porteiros e auxiliares de limpeza. Seus dados, cargo, jornada e remuneração são administrados pelo síndico.

Visitante ou prestador de serviço

Não possui acesso direto ao sistema. Seus dados são registrados pelo porteiro para controlar o horário de entrada, saída, unidade de destino e autorização do morador.

A utilização de perfis diferentes permite limitar o acesso às informações e disponibilizar para cada usuário somente as funções necessárias às suas atividades.

2.2 Wireframes de Baixa Fidelidade

Os wireframes foram criados no Figma para demonstrar a estrutura e o funcionamento inicial das principais telas. Como são protótipos de baixa fidelidade, apresentam poucos elementos decorativos e utilizam formas simples, textos, campos e botões para representar a interface.

Tela 1 - Acesso ao sistema

Apresenta os campos necessários para autenticação. O usuário seleciona seu perfil, informa CPF ou e-mail e senha. A tela também apresenta opções para manter a sessão conectada e recuperar a senha.

Tela 2 - Painel do síndico

Exibe uma visão geral do condomínio. Apresenta saldo em caixa, recebimentos, despesas, chamados de manutenção, ações prioritárias e atividades recentes. Essa tela permite que o síndico acompanhe rapidamente a situação administrativa e financeira.

Tela 3 - Unidades e moradores

Permite consultar as unidades cadastradas, seus respectivos blocos, tipos, moradores responsáveis, situação de ocupação e valor da taxa condominial. Também oferece opções de busca, filtros e cadastro de novas unidades.

Tela 4 - Financeiro

Centraliza pagamentos, receitas e despesas do condomínio. A tela apresenta saldo atual, arrecadação prevista, valores recebidos, valores pendentes e uma tabela com os lançamentos financeiros.

Tela 5 - Portaria e visitantes

Destinada principalmente ao porteiro. Permite registrar visitantes e prestadores, informar documento, unidade de destino, placa do veículo e autorização do morador. Também apresenta as pessoas que estão no condomínio e as autorizações previstas.

Tela 6 - Manutenção

Organiza os chamados de manutenção conforme seu estado, como aberto, em andamento ou concluído. Cada chamado pode apresentar descrição, prioridade, localização, responsável e data de conclusão.

Tela 7 - Funcionários

Permite consultar e cadastrar funcionários. A tela apresenta nome, cargo, contato, jornada, remuneração calculada e situação do colaborador. Também contempla a regra de adicional salarial aplicada ao zelador.

Link público do Figma:
Residencial Golden Java - Sistema de Gestão Condominial

3. Pesquisa Bibliográfica

3.1 Pilares da Programação Orientada a Objetos

A Programação Orientada a Objetos, também conhecida como POO, organiza o desenvolvimento de software a partir de objetos que possuem estado e comportamento. Uma classe funciona como um modelo utilizado para criar objetos, enquanto cada objeto representa uma instância dessa classe.

Segundo Deitel e Deitel (2017), a orientação a objetos contribui para a construção de programas mais organizados, reutilizáveis e fáceis de manter. Entre seus principais fundamentos estão abstração, encapsulamento, herança e polimorfismo. A documentação da Oracle também apresenta objetos como estruturas que reúnem estados e comportamentos relacionados, enquanto as classes funcionam como modelos para sua criação.

Abstração

A abstração consiste em representar somente as características essenciais de uma entidade, deixando de lado detalhes que não são relevantes para o problema tratado.

No sistema Residencial Golden Java, uma unidade pode ser representada pelos atributos número, taxa-base e situação de ocupação. Outras características físicas do imóvel não precisam ser incluídas se não forem utilizadas pelas regras do sistema.

A abstração ajuda a reduzir a complexidade do programa, pois permite modelar cada elemento de acordo com sua função no domínio da aplicação.

Encapsulamento

O encapsulamento consiste em proteger o estado interno de um objeto e controlar a forma como seus dados são acessados ou modificados. Em Java, isso pode ser realizado por meio de atributos private e métodos públicos responsáveis pelas operações permitidas.

A Oracle descreve o encapsulamento como a capacidade de ocultar dados e métodos internos, permitindo que somente os recursos declarados como públicos sejam acessados externamente.

No projeto, a classe Unidade pode manter o valor da taxa como atributo privado. A alteração desse valor deve ocorrer por meio de um método que impeça o cadastro de taxas negativas. Da mesma forma, a classe Condominio deve validar pagamentos e impedir despesas superiores ao saldo disponível.

Assim, as regras de negócio permanecem dentro das classes responsáveis, evitando alterações indevidas em seus atributos.

Herança

A herança permite criar uma nova classe a partir de uma classe existente. A classe derivada recebe atributos e comportamentos da classe mais geral e pode acrescentar ou modificar funcionalidades.

No sistema, a classe Cobertura pode herdar da classe Unidade. Toda cobertura continua sendo uma unidade, mas possui uma regra específica que adiciona 25% ao valor da taxa condominial.

Outro exemplo é a classe Zelador, que pode herdar da classe Funcionario. O zelador possui os dados e comportamentos comuns aos demais funcionários, mas recebe um adicional de 10% no cálculo do salário.

A herança favorece o reaproveitamento de código e ajuda a organizar classes que possuem características em comum. A documentação da Oracle apresenta a herança como um mecanismo para derivar uma classe de outra e reutilizar estados e comportamentos.

Polimorfismo

O polimorfismo permite que objetos relacionados por herança respondam de maneiras diferentes a uma mesma operação. Isso acontece quando uma classe filha redefine um método herdado.

Por exemplo, a classe Unidade possui o método calcularTaxa(). A classe Cobertura pode sobrescrever esse método para acrescentar 25% ao valor normal. Mesmo utilizando uma referência do tipo Unidade, o programa executará o comportamento correspondente ao objeto criado.

O mesmo ocorre com Funcionario e Zelador. Ambos podem possuir o método calcularSalario(), mas o zelador utiliza uma implementação que acrescenta 10% ao salário-base.

Segundo a Oracle, o polimorfismo permite utilizar uma referência mais geral para representar objetos de diferentes subclasses, executando o comportamento correspondente ao tipo real do objeto durante a execução.

Aplicação dos pilares no projeto

Os quatro pilares aparecem de forma integrada no Residencial Golden Java:

- A abstração define quais informações são necessárias para representar moradores, unidades, funcionários e o condomínio.
- O encapsulamento protege os atributos e mantém as validações dentro das classes responsáveis.
- A herança permite criar Cobertura a partir de Unidade e Zelador a partir de Funcionario.
- O polimorfismo permite que métodos como calcularTaxa() e calcularSalario() apresentem comportamentos diferentes conforme o objeto utilizado.

Esses conceitos tornam o sistema mais organizado e facilitam futuras alterações nas regras de negócio.

4. Referências

DEITEL, Paul; DEITEL, Harvey. *Java: como programar*. 10. ed. São Paulo: Pearson Education do Brasil, 2017.

ORACLE. *Lesson: Object-Oriented Programming Concepts*. Java Tutorials. Disponível em: https://docs.oracle.com/javase/tutorial/java/concepts/. Acesso em: 22 set. 2026.

ORACLE. *Inheritance*. Java Tutorials. Disponível em: https://docs.oracle.com/javase/tutorial/java/concepts/inheritance.html. Acesso em: 22 set. 2026.

ORACLE. *Polymorphism*. Java Tutorials. Disponível em: https://docs.oracle.com/javase/tutorial/java/IandI/polymorphism.html. Acesso em: 22 set. 2026.

ORACLE. *Overview of Java*. Java Developer's Guide. Disponível em: https://docs.oracle.com/en/database/oracle/oracle-database/21/jjdev/Java-overview.html. Acesso em: 22 set. 2026.

FIGMA. *Residencial Golden Java - Wireframes de Baixa Fidelidade*. Disponível em: https://www.figma.com/design/ad6bvzg2rHx2nuIZY0wWwB. Acesso em: 22 set. 2026.